

Aqui está um plano de ação dividido em blocos sequenciais. Ao completar cada bloco e conectá-los, você terá o projeto inteiro pronto e dentro dos requisitos obrigatórios.

Bloco 1: Estruturação e Tratamento de Dados

Objetivo: Garantir que o framework consiga ler e limpar diferentes conjuntos de dados de forma padronizada.

- **Ação 1.1:** Escolha e baixe 2 bases de dados textuais (ex: a base de notícias fornecida e um dataset de reviews do Kaggle) e salve-as em uma pasta chamada `data/`.
- **Ação 1.2:** Crie o arquivo `src/ingestion.py`. Escreva funções que leiam essas bases e as transformem em um formato único (por exemplo, um DataFrame do Pandas sempre com as colunas `texto_bruto` e `alvo`).
- **Ação 1.3:** Crie o arquivo `src/preprocessing.py`. Escreva uma função que receba o texto bruto e devolva o texto limpo (transformar em letras minúsculas, remover pontuação e retirar palavras irrelevantes/stopwords).

Bloco 2: Transformação e Algoritmos

Objetivo: Construir as "peças do quebra-cabeça" que serão testadas (representações e modelos).

- **Ação 2.1:** Crie o arquivo `src/embeddings.py`. Implemente duas funções de vetorização: uma básica (como o `TfidfVectorizer` do scikit-learn) e uma avançada (como um modelo do `sentence-transformers`).
- **Ação 2.2:** Crie o arquivo `src/models.py`. Importe e configure 3 algoritmos de classificação simples do scikit-learn (ex: Naive Bayes, Regressão Logística e Random Forest).

Bloco 3: O Orquestrador e o Coração do Projeto (Logs)

Objetivo: Automatizar os testes, juntar as peças dos blocos anteriores e salvar os resultados (A principal exigência do Tema 10).

- **Ação 3.1:** Crie um script chamado `main.py` (ou um notebook `main.ipynb`) que importe as funções dos Blocos 1 e 2.
- **Ação 3.2:** Monte um loop (laço de repetição) que cruze todas as possibilidades: as 2 bases $\times$ os 2 embeddings $\times$ os 3 algoritmos.
- **Ação 3.3:** Dentro desse loop, treine os modelos, calcule a métrica de acerto (Acurácia ou F1-Score) e faça o código **salvar ou atualizar um arquivo** `logs.json` contendo os parâmetros testados e o resultado.

Bloco 4: Serviço Web e Integração com LLM

Objetivo: Criar a API local e adicionar a inteligência externa obrigatória.

- **Ação 4.1:** Crie um arquivo `app.py`. Usando a biblioteca FastAPI, crie uma rota chamada `/predict`.
- **Ação 4.2:** Programe essa rota para receber um texto qualquer enviado pelo usuário. O código da rota deve limpar esse texto, transformá-lo no embedding que teve o melhor resultado no Bloco 3, e usar o melhor modelo para prever a categoria.
- **Ação 4.3:** Antes de devolver a resposta, faça o código enviar a previsão para uma API externa (ex: API do Google Gemini ou OpenAI) com o prompt: *"O modelo classificou este texto como [Categoria]. Explique brevemente o porquê."* Retorne a previsão e a explicação para o usuário.

Bloco 5: Empacotamento e Documentação

Objetivo: Preparar os artefatos finais para entrega e garantir a reprodutibilidade.

- **Ação 5.1:** Crie um arquivo `README.md` explicando de forma direta como instalar as bibliotecas (`pip install -r requirements.txt`), como colocar a chave da API externa e o comando para rodar o FastAPI (`uvicorn app:app --reload`).
- **Ação 5.2:** Escreva o relatório final `.docx` (cerca de 3 a 5 páginas) explicando as escolhas feitas nos blocos 1 e 2 e analisando os resultados salvos no `logs.json`.
- **Ação 5.3:** Crie os slides `.pptx` focando na arquitetura do framework e nos testes automatizados para a apresentação.

Qual desses blocos você gostaria de atacar primeiro para que possamos detalhar a lógica de programação ou as bibliotecas necessárias?

Para solucionar com excelência o **Tema 10 — Framework experimental de NLP com representação intermediária e avaliação reprodutível**, o projeto deve ser abordado com uma mentalidade de engenharia de software e rigor científico. O objetivo principal não é apenas rodar modelos, mas sim construir um ecossistema modular, comparável, documentado e totalmente reaproveitável.

Abaixo está o plano de ação estruturado em fases sequenciais para guiar o desenvolvimento do seu framework dentro de todas as exigências obrigatórias e critérios de valorização do Prof. José Alfredo.

Mapeamento de Componentes vs. Requisitos

Antes de iniciar o código, veja como a arquitetura do seu framework vai cobrir as regras gerais da Unidade 3:

Módulo do Framework PDF+ 1	Requisito Atendido	Detalhes do Escopo PDF
ingestion	Bases de dados	Carga e registro de pelo menos 2 bases textuais distintas.
preprocessing	Pré-processamento	Limpeza, tokenização e isolamento de variáveis de transformação.
embeddings	Representação textual	Comparação de 2 tipos de embeddings (ex: TF-IDF vs. Sentence Transformers).

<code>models</code>	Algoritmos	Execução parametrizada de pelo menos 3 algoritmos .
<code>evaluation & tracking</code>	Métrica e Reprodutibilidade	Geração de representação intermediária (JSON/Logs) e gráficos.
<code>api_service</code>	Uso de API e Artefato	Integração com LLM externa para explicação + Servidor FastAPI.

Plano de Ação Passo a Passo

Fase 1: Concepção, Setup e Ingestão de Dados

O foco inicial é garantir a estabilidade dos dados de entrada e a pergunta central da pesquisa: *como organizar um framework experimental de NLP que seja comparável, documentado e reutilizável?*

1. **Seleção das Bases:** Escolha duas bases textuais públicas (por exemplo, a Planilha de notícias em 6 classes fornecida e uma base do Kaggle ou o *20 Newsgroups*). Documente o motivo da escolha.
2. **Criação do Módulo `ingestion.py`:** Construa funções puras para carregar essas bases, garantindo que o framework converta diferentes formatos de entrada em uma estrutura de dados unificada (ex: um DataFrame do Pandas padronizado com as colunas `texto_bruto` e `target`).

Fase 2: Pipeline de Engenharia de Recursos (Recursos Modulares)

Aqui você construirá os blocos de processamento que serão testados em formato de "cenários experimentais".

1. **Estruturação do `preprocessing.py`:** Crie uma classe ou pipeline configurável que permita ligar/desligar etapas de limpeza (ex: remoção de stopwords, stemming/lemmatização e normalização de caracteres). Isso será crucial para discutir posteriormente o impacto dessas escolhas.
2. **Desenvolvimento do `embeddings.py`:** Implemente uma interface comum para extração de características textuais. Configure e compare no mínimo dois tipos:
 - **Abordagem Clássica:** TF-IDF ou contagem lexical simples.

- **Abordagem Avançada:** Embeddings vetoriais baseados em Transformers (ex: utilizando a biblioteca `sentence-transformers` ou uma API de embeddings de LLM).

Fase 3: Core de Modelagem e a Representação Intermediária

Esta é a assinatura principal do Tema 10. Você precisa criar um motor que execute experimentos de forma automatizada e salve os estados.

1. **Construção do `models.py`:** Configure o framework para aceitar pelo menos 3 algoritmos ou abordagens de Machine Learning (seja para classificação supervisionada ou clustering). Registre os parâmetros centrais de cada um.
2. **Desenvolvimento do Engine de Experimentos:** Crie um laço/orquestrador que combine: `Base de Dados A ou B` $\times$ `Pré-processamento X ou Y` $\times$ `Embedding 1 ou 2` $\times$ `Algoritmo 1, 2 ou 3`.
3. **Persistência da Representação Intermediária:** Cada combinação testada pelo laço deve salvar automaticamente um arquivo **JSON** ou uma linha em uma tabela de **logs padronizada**. Esse arquivo deve conter:
 - Metadados do experimento (ID, data, versão do código).
 - Parâmetros de configuração utilizados.
 - Métricas obtidas (Acurácia, F1-score, ou Silhouette score/métricas de cluster).
 - Caminho para o modelo treinado salvo (se aplicável).
 - Isso dará origem ao seu "caderno de experimentos" automatizado.

Fase 4: Visualização, API e Enriquecimento Semântico

Adicione a camada prática de consumo do framework e a inteligência baseada em serviços externos.

1. **Implementação do `visualization.py`:** Crie funções para ler a sua representação intermediária (JSONs) e gerar automaticamente gráficos comparativos, tabelas de desempenho e matrizes de confusão ou projeções bidimensionais (como PCA/t-SNE).
2. **Integração com API Externa:** Insira uma chamada de API de LLM dentro do pipeline. Use-a estrategicamente na fase de avaliação para: gerar explicações automatizadas sobre os maiores erros do modelo, resumir as classes mais difíceis ou rotular clusters gerados.
3. **Exposição via FastAPI (`api_service.py`):** Crie uma interface web/API local. Crie rotas estruturadas como:
 - `/run_benchmark`: Aciona o pipeline completo e atualiza os logs.
 - `/predict` ou `/analyze`: Recebe um texto inédito, passa pelo pipeline campeão guardado na camada intermediária e retorna a predição junto com a explicação gerada pela API externa.

Fase 5: Documentação Reprodutível e Empacotamento

A entrega final exige um forte caráter científico e de engenharia organizada.

1. **Geração do Relatório Semântico e Apresentação:** Escreva o documento `.docx` e os slides focando nas escolhas metodológicas, análises de erros e limitações encontradas (como latência de API ou perda de sinal semântico no pré-processamento).
2. **Preparação do Repositório (README):** Garanta que o arquivo `README` possua as instruções exatas de reprodutibilidade: como instalar dependências, como configurar as chaves de API e como executar o framework de ponta a ponta.

Estrutura de Pastas Recomendada para o Projeto

Para demonstrar o nível de organização experimental e maturidade computacional esperados, estruture seu repositório da seguinte forma:

Plaintext

- | — data/ # Onde ficam as bases de dados textuais originais [cite: 276]
- | — src/ # Código-fonte modular [cite: 275]
- | | — ingestion.py # Módulo de carga e padronização
- | | — preprocessing.py # Módulo de limpeza e tokenização
- | | — embeddings.py # Extração de TF-IDF e Sentence Transformers
- | | — models.py # Declaração dos 3+ algoritmos avaliados
- | | — visualization.py # Funções para plots, matrizes e projeções
- | — runs/ # Representação Intermediária (O coração do Tema 10)
- | | — experiment_logs.json # Histórico estruturado de todas as execuções
- | | — artifacts/ # Gráficos salvos e tabelas comparativas [cite: 276]
- | — app.py # Servidor FastAPI com as rotas do serviço
- | — main.ipynb # Notebook principal que orquestra os cenários [cite: 20, 275]
- | — Relatorio_U3.docx # Relatório científico curto detalhado [cite: 276]
- | — Apresentacao.pptx # Material visual para o seminário [cite: 276]
- | — README.md # Documentação e manual de reprodutibilidade